1. 200B in bf16. 32 TPU v4p. HBM -> systolic array
$1.2e12$ B/s

$\# \text{bytes} = 2 \cdot 200e9 = 400e9$ bytes

$\text{split} = \dfrac{400e9}{32} = 1.25e10$ per chip

$\text{time} = \dfrac{1.25e10 \text{ bytes}}{1.2e12 \text{ bytes/s}} \approx 0.01\text{s} = 10\text{ms}$

2. • TPU v5e pod 16x16 single host for 8 TPU (4 x 2)

$\# \text{host} = \dfrac{16 \cdot 16}{8} = 32$ # cores = 1 per chip = 256

$\# \text{FLOPs/s pod} = 1.97e14 \cdot 256 = 5.0432e16$ FLOPs/s

total HBM $= 16 \cdot 256 = 4096$ GB

• TPU v5p pod 16x20x28 1 host for (2 x 2 x 1)

$\# \text{host} = \dfrac{16 \cdot 20 \cdot 28}{4} = 2240$ # cores = 2 per chip = 17920

$\# \text{FLOPs pod} = 4.59e14 \cdot 8960 = 4.11264e18$ FLOPs/s

total HBM $= 8960 \cdot 96 = 860160$ GB

③ $A \sim bf16[D, F]$ $x \sim bf16[B, D]$ in host DRAM

TPU v6e. $B \ll D$, $F = 4D$ PCIe BW $= 1.6e10$ B/s

$$\# \text{bytes} = 2(D \cdot F + B \cdot D + BF)$$

$$T_{math} = \frac{2BDF}{9.18e14} > T_{comm} = \frac{2(DF + BD + BF)}{1.6e10} s$$

$$\Downarrow$$

$$\frac{\cancel{8}BD^{\cancel{2}}}{9.18e14} > \frac{\cancel{8}D^{\cancel{2}} + \cancel{10BD}}{1.6e10} \to \text{we } B \ll D$$

$$\Downarrow$$

$$B > \frac{9.18e14}{1.6e10} \approx 57375$$

→ intensity of v6e with respect to PCIe.

④

$A \sim int8[16384, 4096]$ $x \sim int8[B, 4096]$ TPU v5e

$\# \text{bytes} = 67108864 + 20480B$ VMEM 128 MiB

VMEM BW 22 · HBM BW

(check the plots in 4. ipynb)

① Even though we can load all in the VMEM, we still pay the full price if we need to do just 1 matmul.

$$T_{math} = \frac{B \cdot 134217728}{3.93e14} \qquad T_{comm} = \frac{67108864 + 20480B}{8e11}$$

load to VMEM

$T_{lower} = \max(T_{math}, T_{comm})$ (assuming max overlap between load and compute)

$$\frac{B \cdot 134217728}{3.93e14} > \frac{67108864 + 20480B}{8e11}$$

$$\Downarrow$$

$$B = 266$$

② If the data are in VMEM, the BW is 22 times greater.

$B = 12$

⑤ V5e 4x4 slice bf16[8, 128, 8192] [0,0] -> [3,3]

per hop latency 1 μs. ICI = 4.5e10 bytes/s

① We have 6 hops so the first byte is after 6 μs.

② # bytes = $8 \cdot 128 \cdot 8192 \cdot 2 = 16777216 \simeq 1.7e7$

We can split the # bytes in 2 to be sent using 2 comm ports.

$2 \cdot 4.5e10 = 9e10$

$$T_{comm} = \frac{1.7e7}{9e10} = 189 \ \mu s$$

Total time $189 + 6 = 195 \ \mu s$

⑥ A ~ i8[128·1024, 128·1024] sharded evenly a TPU v5e 4x4, offloaded to host DRAM on each chip. Copy A to TPU[0,0] and mul by x ~ bf16[8, 128·1024]. 2 hosts H_1 H_2 ICI =

$PCIe = 1.6e10$ bytes/s $\qquad$ $DCN = 6.25e9$ bytes/s

$\#$ bytes $A = 128 \cdot 1024 \cdot 128 \cdot 1024 = 17179869184 \simeq 1.72e10$ bytes

$\#$ bytes $x = 2 \cdot 8 \cdot 128 \cdot 1024 = 2097152 \simeq 2.1e6$ bytes

1 These bytes are in the 2 hosts DRAM. The 2 hosts communicate through DCN, while host <-> slice is PCIe. To load everything in TPU [0,0], we first send $\frac{1.72e10}{2}$ from H_2 to H_1.

$$T_{comm}(A/2)_{H_2 \to H_1} = \frac{1.72e10/2}{6.25e9} = 1.4 \text{ s}$$

$$T_{comm}(A)_{H \to T[0,0]} = \frac{1.72e10}{1.6e10} = 1.1 \text{ s}$$

2 Another approach is to load A to each corresponding slice and then use ICI to transfer everything to T[0,0]. This avoid DCN.

$$T_{comm}(A/2)_{H_i \to T} = \frac{1.72e10/2/8}{1.6e10} = 0.07 \text{ s}$$

One portion of A is already in T[0,0] with this method, so we need to transfer the other 15.

$$T_{comm_{ICI}} = \frac{1.72e10 - \left(\frac{1.72e10}{16}\right)}{\underset{\text{(exercise 5)}}{\overset{?}{9e10}}} = 0.179 \text{ s}$$

This second method is better.

$$T_{comm_{HBM \to VMEM}} = \frac{1.72e10 + \overset{\to x \text{ is small can be ignored}}{2.16e6}}{8.2e11} = 0.021 \text{ s}$$

$$T_{math} = \frac{2 \cdot 8 \cdot (128 \cdot 1024)(128 \cdot 1024)}{1.97e14} = 0.001 \text{ s}$$

We can take as an upper bound (meaning no overlap of mem fetch and computation) the sum of all the times:

$$0.07 + 0.179 + 0.021 + 0.001 = 0.271 \text{ s}$$